

SHA-256 Geometric / Algebraic Companion Notebook

This notebook is a **runnable companion** to the current SHA-256 trust-audit paper.

It demonstrates, in code:

1. the sparse SHA-256 round geometry,
2. exact local reverse closure up to the fused wall,
3. final-add carry restoration,
4. the Sziklai differential invariant,
5. admissible side-geometry bundle extraction,
6. one-round and multi-round scoring on real Bitcoin headers,
7. best-first predecessor-fiber navigation through bounded depths,
8. the current fused-wall split analysis.

Proof boundary

This notebook demonstrates **what is implemented and verified so far**.

It does **not** claim:

- a blind 64-round SHA-256 preimage break,
- a Bitcoin mining shortcut,
- a proof that the current bundle is globally injective,
- a true A* search with a proven admissible future-cost lower bound.

```
In [1]: # Runtime / install cell
# This notebook is designed to run with the Python standard library only.
# Optional if you want to add your own charts later:
# %pip install matplotlib pandas
```

```
In [2]: from __future__ import annotations

from dataclasses import dataclass
from typing import Dict, List, Tuple, Iterable
import hashlib
import random
import statistics
import struct
import time

MASK32 = 0xFFFFFFFF

H0 = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

K = [
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90bffffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2,
]

def u32(x: int) -> int:
    return x & MASK32

def rotr(x: int, n: int) -> int:
    x &= MASK32
    return ((x >> n) | (x << (32 - n))) & MASK32

def ch(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (~x & z)) & MASK32

def maj(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (x & z) ^ (y & z)) & MASK32
```

```

def Sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def hw(x: int) -> int:
    return (x & MASK32).bit_count()

def as_hex32(x: int) -> str:
    return f"0x{x & MASK32:08x}"

def words_to_hex(words: Iterable[int]) -> List[str]:
    return [as_hex32(w) for w in words]

def chunked(seq, n):
    seq = list(seq)
    return [seq[i:i+n] for i in range(0, len(seq), n)]

def add32(a: int, b: int) -> Tuple[int, int]:
    total = (a & MASK32) + (b & MASK32)
    return total & MASK32, int(total >> 32)

def carry_mask_add(a: int, b: int) -> int:
    a &= MASK32
    b &= MASK32
    carry = a & b
    union = carry
    s = a ^ b
    while carry:
        carry = (carry << 1) & MASK32
        newcarry = s & carry
        union |= newcarry
        s ^= carry
        carry = newcarry
    return union

```

```

def nibble_hws(x: int) -> Tuple[int, ...]:
    return tuple(hw((x >> shift) & 0xF) for shift in range(0, 32, 4))

def chirality_split(x: int) -> Tuple[int, int]:
    even_mask = 0x55555555
    odd_mask = 0xAAAAAAAA
    return hw(x & even_mask), hw(x & odd_mask)

def carry_span(mask: int) -> int:
    mask &= MASK32
    best = cur = 0
    for i in range(32):
        if (mask >> i) & 1:
            cur += 1
            best = max(best, cur)
        else:
            cur = 0
    return best

def first_last_hit(mask: int) -> Tuple[int, int]:
    mask &= MASK32
    if mask == 0:
        return (-1, -1)
    first = min(i for i in range(32) if (mask >> i) & 1)
    last = max(i for i in range(32) if (mask >> i) & 1)
    return first, last

def pad_sha256(msg: bytes) -> bytes:
    bit_len = len(msg) * 8
    out = msg + b"\x80"
    while len(out) % 64 != 56:
        out += b"\x00"
    out += struct.pack(">Q", bit_len)
    return out

def words_from_block(block: bytes) -> List[int]:
    return list(struct.unpack(">16I", block))

def expand_schedule(w16: List[int]) -> List[int]:
    W = list(w16)
    for t in range(16, 64):

```

```

        W.append(u32(sigma1(W[t - 2]) + W[t - 7] + sigma0(W[t - 15]) + W[t - 16]))
    return W

def dbl_sha256_hex_display(msg: bytes) -> str:
    return hashlib.sha256(hashlib.sha256(msg).digest()).digest()[::-1].hex()

@dataclass
class RoundTrace:
    t: int
    a: int
    b: int
    c: int
    d: int
    e: int
    f: int
    g: int
    h: int
    Wt: int
    T1: int
    T2: int
    stage_carries: Tuple[int, int, int, int]
    stage_masks: Tuple[int, int, int, int]
    h_hw: int
    h_nibble_hw: Tuple[int, ...]
    h_chirality: Tuple[int, int]
    stage_spans: Tuple[int, int, int, int]

def compress_block_trace(block: bytes, state: List[int]) -> Dict[str, object]:
    W = expand_schedule(words_from_block(block))
    a, b, c, d, e, f, g, h = state
    traces: List[RoundTrace] = []

    for t in range(64):
        s1 = Sigma1(e)
        chv = ch(e, f, g)

        sA, c1 = add32(h, s1)
        sB, c2 = add32(sA, chv)
        sC, c3 = add32(sB, K[t])
        T1, c4 = add32(sC, W[t])

        m1 = carry_mask_add(h, s1)

```

```

m2 = carry_mask_add(sA, chv)
m3 = carry_mask_add(sB, K[t])
m4 = carry_mask_add(sC, W[t])

T2 = u32(Sigma0(a) + maj(a, b, c))

traces.append(
    RoundTrace(
        t=t, a=a, b=b, c=c, d=d, e=e, f=f, g=g, h=h,
        Wt=W[t], T1=T1, T2=T2,
        stage_carries=(c1, c2, c3, c4),
        stage_masks=(m1, m2, m3, m4),
        h_hw=hw(h),
        h_nibble_hw=nibble_hws(h),
        h_chirality=chirality_split(h),
        stage_spans=(carry_span(m1), carry_span(m2), carry_span(m3), carry_span(m4)),
    )
)

new_a = u32(T1 + T2)
new_e = u32(d + T1)
a, b, c, d, e, f, g, h = new_a, a, b, c, new_e, e, f, g

working_final = [a, b, c, d, e, f, g, h]
state_out = [u32(state[i] + v) for i, v in enumerate(working_final)]
return {
    "W": W,
    "traces": traces,
    "working_final": working_final,
    "state_out": state_out,
}

def sha256_trace_full(msg: bytes) -> List[Dict[str, object]]:
    padded = pad_sha256(msg)
    blocks = [padded[i:i+64] for i in range(0, len(padded), 64)]
    state = H0[:]
    out = []
    for idx, block in enumerate(blocks):
        step = compress_block_trace(block, state)
        out.append({
            "block_index": idx,
            "block": block,

```

```

        "init_state": state[:],
        **step,
    })
    state = step["state_out"]
    return out

def sha256_digest_words(msg: bytes) -> List[int]:
    return list(struct.unpack(">8I", hashlib.sha256(msg).digest()))

print("Loaded SHA-256 utilities.")

```

Loaded SHA-256 utilities.

Real Bitcoin headers used in the notebook

- **Genesis block**
- **Block 328,734** (reference header often used in Bitcoin docs)

The notebook works on the **second compression block of the first SHA-256 pass** for the 80-byte header, because that is the mining-relevant block where the timestamp / bits / nonce live.

```

In [3]: GENESIS_HEADER_HEX = (
    "01000000"
    + "00" * 32
    + "3ba3edfd7a7b12b27ac72c3e67768f617fc81bc3888a51323a9fb8aa4b1e5e4a"
    + "29ab5f49"
    + "ffff001d"
    + "1dac2b7c"
)

BLOCK_328734_HEADER_HEX = (
    "02000000"
    "b6ff0b1b1680a2862a30ca44d346d9e8"
    "910d334beb48ca0c000000000000000"
    "9d10aa52ee949386ca9385695f04ede2"
    "70dda20810decd12bc9b048aaab31471"
    "24d95a54"
    "30c31b18"
    "fe9f0864"
)

```

```

REAL_HEADERS = {
    "genesis": {
        "height": 0,
        "known_hash": "00000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f",
        "header": bytes.fromhex(GENESIS_HEADER_HEX),
    },
    "block_328734": {
        "height": 328734,
        "known_hash": "00000000000000009a11b3972c8e532fe964de937c9e0096b43814e67af3728",
        "header": bytes.fromhex(BLOCK_328734_HEADER_HEX),
    },
}

def second_block_of_first_sha(header: bytes) -> Dict[str, object]:
    traced = sha256_trace_full(header)
    assert len(traced) == 2, "80-byte Bitcoin header should pad to two 64-byte blocks in first SHA pass."
    return traced[1]

def verify_bitcoin_headers() -> Dict[str, object]:
    out = {}
    for name, item in REAL_HEADERS.items():
        got = dbl_sha256_hex_display(item["header"])
        out[name] = {
            "hash_ok": got == item["known_hash"],
            "known_hash": item["known_hash"],
            "computed_hash": got,
            "second_block_first_words": words_to_hex(words_from_block(second_block_of_first_sha(item["header"]))["block"])
        }
    return out

verify_bitcoin_headers()

```



```
Out[3]: {'genesis': {'hash_ok': True,
  'known_hash': '00000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f',
  'computed_hash': '00000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f',
  'second_block_first_words': ['0x4b1e5e4a',
  '0x29ab5f49',
  '0xfffff001d',
  '0x1dac2b7c']},
  'block_328734': {'hash_ok': True,
  'known_hash': '00000000000000009a11b3972c8e532fe964de937c9e0096b43814e67af3728',
  'computed_hash': '00000000000000009a11b3972c8e532fe964de937c9e0096b43814e67af3728',
  'second_block_first_words': ['0xaab31471',
  '0x24d95a54',
  '0x30c31b18',
  '0xfe9f0864']}}}
```

Exact local reverse closure and the fused wall

This section shows the current central algebraic fact.

Given the **next round state** x_{t+1} , most of the predecessor state closes exactly. The only unresolved object is the fused split $F_t \equiv h_t + W_t \pmod{2^{32}}$.

```
In [4]: def reverse_knowns_from_next(next_state: List[int], t: int) -> Dict[str, object]:
    a1, b1, c1, d1, e1, f1, g1, h1 = next_state

    a_t = b1
    b_t = c1
    c_t = d1
    e_t = f1
    f_t = g1
    g_t = h1

    T2 = u32(Sigma0(a_t) + maj(a_t, b_t, c_t))
    T1 = u32(a1 - T2)
    d_t = u32(e1 - T1)

    fused_F = u32(T1 - Sigma1(e_t) - ch(e_t, f_t, g_t) - K[t])

    return {
```

```

    "a_t": a_t,
    "b_t": b_t,
    "c_t": c_t,
    "d_t": d_t,
    "e_t": e_t,
    "f_t": f_t,
    "g_t": g_t,
    "T1": T1,
    "T2": T2,
    "F_t": fused_F,
}

```

```

def reverse_step_from_next(next_state: List[int], t: int, W_guess: int) -> Dict[str, object]:
    known = reverse_knowns_from_next(next_state, t)

```

```

    a_t = known["a_t"]
    b_t = known["b_t"]
    c_t = known["c_t"]
    d_t = known["d_t"]
    e_t = known["e_t"]
    f_t = known["f_t"]
    g_t = known["g_t"]
    T1 = known["T1"]
    T2 = known["T2"]

```

```

    h_t = u32(known["F_t"] - W_guess)

```

```

    s1 = Sigma1(e_t)
    chv = ch(e_t, f_t, g_t)
    sA, c1 = add32(h_t, s1)
    sB, c2 = add32(sA, chv)
    sC, c3 = add32(sB, K[t])
    T1_check, c4 = add32(sC, W_guess)

```

```

    return {
        "state": [a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t],
        "stage_carries": (c1, c2, c3, c4),
        "T1": T1,
        "T2": T2,
        "T1_check": T1_check,
        "F_t": known["F_t"],
    }

```

```

# Demonstrate on genesis, round 63, second block of first SHA pass
g_block = second_block_of_first_sha(REAL_HEADERS["genesis"]["header"])
g_trace = g_block["traces"]
t = 63
next_state = g_block["working_final"] # this is  $x_{64}$ , so for  $t=63$  it is  $x_{t+1}$ 

known = reverse_knowns_from_next(next_state, t)
truth = reverse_step_from_next(next_state, t, g_block["W"][t])

print("Round:", t)
print("True W[t]:", as_hex32(g_block["W"][t]))
print("Fused F[t]:", as_hex32(known["F_t"]))
print("Recovered h[t] from true W[t]:", as_hex32(truth["state"][7]))
print("Trace h[t]:", as_hex32(g_trace[t].h))
print("Exact predecessor match:", truth["state"] == [g_trace[t].a, g_trace[t].b, g_trace[t].c, g_trace[t].d, g_trace[t].e, g_trace[t].f, g_trace[t].g, g_trace[t].h])

```

```

Round: 63
True W[t]: 0x86b0b8d5
Fused F[t]: 0x7da29f3f
Recovered h[t] from true W[t]: 0xf6f1e66a
Trace h[t]: 0xf6f1e66a
Exact predecessor match: True

```

Final-add carry restoration identity

This demonstrates the exact restoration of the pre-feed-forward terminal state from an observed digest and known input state for a single-block SHA-256 example.

```

In [5]: def final_add_carry_vector(digest_words: List[int], iv_words: List[int]) -> List[int]:
        return [int(h < iv) for h, iv in zip(digest_words, iv_words)]

def final_add_restore_state(digest_words: List[int], iv_words: List[int]) -> List[int]:
    k = final_add_carry_vector(digest_words, iv_words)
    return [digest_words[i] - iv_words[i] + (k[i] << 32) for i in range(8)]

msg = b"abc"
trace_abc = sha256_trace_full(msg)
assert len(trace_abc) == 1, "abc should fit in one padded block"

```

```

working_final = trace_abc[0]["working_final"]
digest_words = trace_abc[0]["state_out"]
restored = final_add_restore_state(digest_words, H0)

print("Digest words:", words_to_hex(digest_words))
print("Restored pre-feed-forward state matches working_final:", restored == working_final)
print("Carry vector:", final_add_carry_vector(digest_words, H0))

```

Digest words: ['0xba7816bf', '0x8f01cfea', '0x414140de', '0x5dae2223', '0xb00361a3', '0x96177a9c', '0xb410ff61', '0xf20015ad']

Restored pre-feed-forward state matches working_final: True

Carry vector: [0, 1, 0, 1, 0, 1, 0, 0]

Sziklai differential invariant

We verify $a_{t+1} - e_{t+1} \equiv T2_t - d_t \pmod{2^{32}}$ over many random test messages and all traced rounds.

```

In [6]: def verify_sziklai(num_messages: int = 200, max_len: int = 96, seed: int = 12345) -> Dict[str, int]:
    rng = random.Random(seed)
    violations = 0
    checked = 0
    for _ in range(num_messages):
        msg = bytes(rng.getrandbits(8) for _ in range(rng.randrange(0, max_len + 1)))
        for block in sha256_trace_full(msg):
            traces = block["traces"]
            # We have x_t directly in each trace; reconstruct x_{t+1} from the next trace or final working state.
            for i, tr in enumerate(traces):
                if i < 63:
                    nxt = traces[i + 1]
                    a_next, e_next = nxt.a, nxt.e
                else:
                    wf = block["working_final"]
                    a_next, e_next = wf[0], wf[4]
                lhs = u32(a_next - e_next)
                rhs = u32(tr.T2 - tr.d)
                checked += 1
                if lhs != rhs:
                    violations += 1
    return {"checked_rounds": checked, "violations": violations}

```

```
verify_sziklai()
```

```
Out[6]: {'checked_rounds': 18176, 'violations': 0}
```

Admissible geometry bundle

This notebook currently uses a bundle built from:

- staged carry bits,
- full staged carry masks,
- NOP-subtracted carry masks,
- chirality splits,
- nibble silhouettes of h_t ,
- carry-span witnesses.

This remains **side geometry**. It does not export message words directly.

```
In [7]: def nop_backbone_masks_for_round(t: int, e_t: int, f_t: int, g_t: int) -> Tuple[int, int, int, int]:  
    # message-free local backbone for the T1-side staged additions, holding W[t] = 0  
    h_t = 0  
    s1 = Sigma1(e_t)  
    chv = ch(e_t, f_t, g_t)  
    sA, _ = add32(h_t, s1)  
    sB, _ = add32(sA, chv)  
    sC, _ = add32(sB, K[t])  
    return (  
        carry_mask_add(h_t, s1),  
        carry_mask_add(sA, chv),  
        carry_mask_add(sB, K[t]),  
        carry_mask_add(sC, 0),  
    )  
  
def build_bundle_entry(trace: RoundTrace) -> Dict[str, object]:  
    nop_masks = nop_backbone_masks_for_round(trace.t, trace.e, trace.f, trace.g)  
    nop_sub = tuple((a ^ b) & MASK32 for a, b in zip(trace.stage_masks, nop_masks))  
    return {  
        "stage_carries": trace.stage_carries,
```

```

    "stage_masks": trace.stage_masks,
    "nop_sub_masks": nop_sub,
    "mask_hw": tuple(hw(x) for x in trace.stage_masks),
    "nop_sub_hw": tuple(hw(x) for x in nop_sub),
    "mask_chirality": tuple(chirality_split(x) for x in trace.stage_masks),
    "nop_sub_chirality": tuple(chirality_split(x) for x in nop_sub),
    "h_hw": trace.h_hw,
    "h_nibble_hw": trace.h_nibble_hw,
    "h_chirality": trace.h_chirality,
    "stage_spans": trace.stage_spans,
    "stage_first_last": tuple(first_last_hit(x) for x in trace.stage_masks),
}

```

```

def score_candidate(traces: List[RoundTrace], next_state: List[int], t: int, guess: int) -> Dict[str, int]:
    obs_trace = traces[t]
    obs = build_bundle_entry(obs_trace)
    pred = reverse_step_from_next(next_state, t, guess)
    a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t = pred["state"]

    s1 = Sigma1(e_t)
    chv = ch(e_t, f_t, g_t)
    sA, _ = add32(h_t, s1)
    sB, _ = add32(sA, chv)
    sC, _ = add32(sB, K[t])

    pred_masks = (
        carry_mask_add(h_t, s1),
        carry_mask_add(sA, chv),
        carry_mask_add(sB, K[t]),
        carry_mask_add(sC, guess),
    )
    nop_masks = nop_backbone_masks_for_round(t, e_t, f_t, g_t)
    pred_nop_sub = tuple((a ^ b) & MASK32 for a, b in zip(pred_masks, nop_masks))

    carry_mismatch = sum(int(a != b) for a, b in zip(pred["stage_carries"], obs["stage_carries"]))
    mask_bit_mismatch = sum(hw(a ^ b) for a, b in zip(pred_masks, obs["stage_masks"]))
    nop_sub_bit_mismatch = sum(hw(a ^ b) for a, b in zip(pred_nop_sub, obs["nop_sub_masks"]))
    h_hw_error = abs(hw(h_t) - obs["h_hw"])
    h_nibble_error = sum(abs(a - b) for a, b in zip(nibble_hws(h_t), obs["h_nibble_hw"]))
    h_chirality_error = sum(abs(a - b) for a, b in zip(chirality_split(h_t), obs["h_chirality"]))
    span_error = sum(abs(carry_span(a) - b) for a, b in zip(pred_masks, obs["stage_spans"]))

```

```

total = (
    5 * carry_mismatch
    + mask_bit_mismatch
    + nop_sub_bit_mismatch
    + h_hw_error
    + h_nibble_error
    + h_chirality_error
    + span_error
)

return {
    "score": total,
    "carry_mismatch": carry_mismatch,
    "mask_bit_mismatch": mask_bit_mismatch,
    "nop_sub_bit_mismatch": nop_sub_bit_mismatch,
    "h_hw_error": h_hw_error,
    "h_nibble_error": h_nibble_error,
    "h_chirality_error": h_chirality_error,
    "span_error": span_error,
}

# One-round score check on real data
for name, item in REAL_HEADERS.items():
    block = second_block_of_first_sha(item["header"])
    t = 63
    true_w = block["W"][t]
    true_score = score_candidate(block["traces"], block["working_final"], t, true_w)["score"]
    rng = random.Random(20260412 + block["W"][t])
    sample_scores = [score_candidate(block["traces"], block["working_final"], t, rng.getrandbits(32))["score"] for _
    print(name)
    print(" true W63:", as_hex32(true_w))
    print(" true score:", true_score)
    print(" best random score:", min(sample_scores))
    print(" median random score:", statistics.median(sample_scores))

```

```
genesis
  true W63: 0x86b0b8d5
  true score: 0
  best random score: 62
  median random score: 128.0
block_328734
  true W63: 0xa572aedd
  true score: 0
  best random score: 56
  median random score: 118.0
```

Local candidate generation

This is not a full 2^{32} search. It is a local, geometry-guided candidate generator:

- nibble descent,
- optional bit-flip refinement,
- bundle scoring.

That is enough to demonstrate the **ranked navigation** currently implemented.

```
In [8]: def nibble_descent(block: Dict[str, object], next_state: List[int], t: int, start: int, sweeps: int = 8) -> Tuple[int, int]:
    cur = start & MASK32
    cur_s = score_candidate(block["traces"], next_state, t, cur)["score"]

    for _ in range(sweeps):
        improved = False
        for pos in range(8):
            shift = pos * 4
            base = cur & ~(0xF << shift)
            best_word, best_score = cur, cur_s
            for nib in range(16):
                cand = base | (nib << shift)
                s = score_candidate(block["traces"], next_state, t, cand)["score"]
                if s < best_score or (s == best_score and cand < best_word):
                    best_word, best_score = cand, s
            if best_word != cur:
                cur, cur_s = best_word, best_score
                improved = True
```



```

        if not improved:
            break

    return cur, cur_s

def local_candidate_pool(block: Dict[str, object], next_state: List[int], t: int, restarts: int = 48, keep: int = 12):
    rng = random.Random(seed)
    starts = [0, MASK32, 0xAAAAAAAA, 0x55555555] + [rng.getrandbits(32) for _ in range(restarts)]
    best: Dict[int, int] = {}

    for start in starts:
        w, s = nibble_descent(block, next_state, t, start)
        improved = True
        while improved:
            improved = False
            for bit in range(32):
                cand = w ^ (1 << bit)
                sc = score_candidate(block["traces"], next_state, t, cand)["score"]
                if sc < s or (sc == s and cand < w):
                    w, s = cand, sc
                    improved = True
            best[w] = min(best.get(w, 10**9), s)

    rows = sorted((s, w) for w, s in best.items())[:keep]
    return rows

for name, item in REAL_HEADERS.items():
    block = second_block_of_first_sha(item["header"])
    rows = local_candidate_pool(block, block["working_final"], 63, restarts=32, keep=10, seed=42)
    print(f"\n{name} top local pool candidates for round 63")
    for rank, (s, w) in enumerate(rows, 1):
        marker = " <== true" if w == block["W"][63] else ""
        print(f"{rank:2d}. score={s:3d} W63={as_hex32(w)}{marker}")

```

genesis top local pool candidates for round 63

1. score= 0 W63=0x86b0b8d5 <== true
2. score= 6 W63=0x86b0b914
3. score= 14 W63=0x86acb915

block_328734 top local pool candidates for round 63

1. score= 0 W63=0xa572aedd <== true
2. score= 4 W63=0x8532aedd
3. score= 4 W63=0x8572aadd

Best-first predecessor-fiber navigation

This is the current bounded-depth multi-round search:

- no future-cost heuristic,
- accumulated residual only,
- exact local reverse closure at each step,
- local candidate pool per round.

So this is **best-first** rather than true A*.

In [9]: `import heapq`

```
def best_first_search(block: Dict[str, object], t_hi: int = 63, t_lo: int = 60,
                      max_expansions: int = 1500, restarts: int = 32, keep_local: int = 3,
                      seed: int = 42) -> Tuple[List[Dict[str, object]], int]:
    rng = random.Random(seed)
    pq = []
    counter = 0
    heapq.heappush(pq, (0, counter, tuple(block["working_final"]), t_hi, tuple(), tuple()))
    best_state_cost = {}
    completed = []
    expansions = 0

    while pq and expansions < max_expansions and len(completed) < 20:
        g, _, next_state_tup, t, guesses_tup, per_tup = heapq.heappop(pq)
        if t < t_lo:
            completed.append({
                "total": g,
```

```

        "guesses": dict(guesses_tup),
        "per_round": list(per_tup),
    })
    continue

    dom = (t, next_state_tup)
    prev = best_state_cost.get(dom)
    if prev is not None and prev <= g:
        continue
    best_state_cost[dom] = g
    expansions += 1

    rows = local_candidate_pool(block, list(next_state_tup), t,
                                restarts=restarts, keep=keep_local,
                                seed=rng.randrange(1 << 30))

    for s, w in rows:
        pred = reverse_step_from_next(list(next_state_tup), t, w)
        counter += 1
        guesses = tuple(sorted(**dict(guesses_tup), t: w).items()))
        per = tuple(list(per_tup) + [(t, s, w)])
        heapq.heappush(pq, (g + s, counter, tuple(pred["state"]), t - 1, guesses, per))

    completed = sorted(completed, key=lambda p: (p["total"], tuple(sorted(p["guesses"].items()))))
    return completed, expansions

def as_hex_chain(guesses: Dict[int, int]) -> Dict[int, str]:
    return {t: as_hex32(guesses[t]) for t in sorted(guesses)}

```

In [10]: RUN_DEEP_SEARCH = False

```

def run_depths(depths=(4,), restarts=16, keep_local=2, max_expansions=1200):
    summary = []
    for depth in depths:
        t_lo = 64 - depth
        print(f"\n=== depth {depth} (rounds 63..{t_lo}) ===")
        for name, item in REAL_HEADERS.items():
            block = second_block_of_first_sha(item["header"])
            truth = {t: block["W"][t] for t in range(t_lo, 64)}
            t0 = time.time()
            completed, expansions = best_first_search(
                block, 63, t_lo,
                max_expansions=max_expansions,

```

```

        restarts=restarts,
        keep_local=keep_local,
        seed=42,
    )
    dt = time.time() - t0
    rank = None
    best_false = None
    for i, p in enumerate(completed, 1):
        if p["guesses"] == truth:
            rank = i
            elif best_false is None:
                best_false = p["total"]
    summary.append({
        "name": name,
        "depth": depth,
        "rank": rank,
        "expansions": expansions,
        "time_sec": round(dt, 3),
        "true_total": next((p["total"] for p in completed if p["guesses"] == truth), None),
        "best_false_total": best_false,
    })
    print(f"{name}: rank={rank}, expansions={expansions}, time={dt:.3f}s")
    if completed:
        for i, p in enumerate(completed[:3], 1):
            print(f"  top{i}: total={p['total']} chain={as_hex_chain(p['guesses'])}")

    return summary

quick_depths = (4,) if not RUN_DEEP_SEARCH else (4, 6, 8)
depth_summary = run_depths(depths=quick_depths)
depth_summary

```

=== depth 4 (rounds 63..60) ===

genesis: rank=1, expansions=15, time=6.946s

top1: total=0 chain={60: '0xfd9e331a', 61: '0xd86c39a0', 62: '0x61cdb5cb', 63: '0x86b0b8d5'}

top2: total=4 chain={60: '0xbd9e131a', 61: '0xd86c39a0', 62: '0x61cdb5cb', 63: '0x86b0b8d5'}

top3: total=4 chain={60: '0xfd96331a', 61: '0xd86439a8', 62: '0x61cdb5cb', 63: '0x86b0b8d5'}

block_328734: rank=1, expansions=15, time=7.094s

top1: total=0 chain={60: '0x735e832a', 61: '0x6bebedd6', 62: '0xf444e8df', 63: '0xa572aedd'}

top2: total=2 chain={60: '0x537e832a', 61: '0x6bebedd6', 62: '0xf444e8df', 63: '0xa572aedd'}

top3: total=8 chain={60: '0x735e83aa', 61: '0x6bf3ed56', 62: '0xf444e8df', 63: '0xa572aedd'}

```
Out[10]: [{'name': 'genesis',
          'depth': 4,
          'rank': 1,
          'expansions': 15,
          'time_sec': 6.946,
          'true_total': 0,
          'best_false_total': 4},
         {'name': 'block_328734',
          'depth': 4,
          'rank': 1,
          'expansions': 15,
          'time_sec': 7.094,
          'true_total': 0,
          'best_false_total': 2}]
```

Optional deeper search

The next cell is intentionally **not** run by default in the executed notebook.

Set larger search budgets only when you want to profile the current bundle at depths 6 and 8.

```
In [11]: # Optional deeper run
# Uncomment to push the current search further.
#
# deep_summary = run_depths(
#     depths=(4, 6, 8),
#     restarts=24,
#     keep_local=3,
#     max_expansions=1800,
# )
# deep_summary
```

Current fused-wall split analysis

This section does **not** fully solve the split $F_t \equiv h_t + W_t \pmod{2^{32}}$.

It shows the current bitwise finite-state structure.

If

- F_j is bit j of F_t ,
- h_j and w_j are the split bits,
- c_j is the carry into bit j ,

then $h_j + w_j + c_j = F_j + 2c_{j+1}$, so $w_j = F_j \oplus h_j \oplus c_j$.

The carry recursion is $c_{j+1} = \begin{cases} h_j, & \text{if } F_j = c_j \\ c_j, & \text{if } F_j \neq c_j \end{cases}$

This means the fused wall is already a constrained bitwise decoder, not an opaque 32-bit atom.

```
In [12]: def add_carry_trace(a: int, b: int) -> Tuple[int, List[int]]:
    a &= MASK32
    b &= MASK32
    carry = 0
    carries = [carry]
    total = 0
    for j in range(32):
        aj = (a >> j) & 1
        bj = (b >> j) & 1
        s = aj ^ bj ^ carry
        total |= (s << j)
        carry = (aj & bj) | (aj & carry) | (bj & carry)
        carries.append(carry)
    return total & MASK32, carries # carries[j] = carry into bit j

def fused_wall_forcing_stats(F: int, carries: List[int]) -> Dict[str, object]:
    forced_bits = []
    ambiguous_bits = []
    recovered_h = 0
    recovered_w = 0

    for j in range(32):
        F_j = (F >> j) & 1
        c_j = carries[j]
        c_next = carries[j + 1]
        if F_j == c_j:
            h_j = c_next
            w_j = F_j ^ h_j ^ c_j
```

```

        recovered_h |= (h_j << j)
        recovered_w |= (w_j << j)
        forced_bits.append(j)
    else:
        ambiguous_bits.append(j)

    return {
        "forced_count": len(forced_bits),
        "ambiguous_count": len(ambiguous_bits),
        "forced_bits": forced_bits,
        "ambiguous_bits": ambiguous_bits,
        "partial_h": recovered_h,
        "partial_w": recovered_w,
    }

# Demonstrate on real round data using the true split only to inspect the forcing pattern.
for name, item in REAL_HEADERS.items():
    block = second_block_of_first_sha(item["header"])
    t = 63
    tr = block["traces"][t]
    F_t = reverse_knowns_from_next(block["working_final"], t)["F_t"]
    total, carries = add_carry_trace(tr.h, tr.Wt)
    stats = fused_wall_forcing_stats(F_t, carries)
    print(name)
    print(" F_t matches h_t + W_t:", total == F_t)
    print(" forced bits from carry trace alone:", stats["forced_count"])
    print(" ambiguous bits remaining:", stats["ambiguous_count"])
    print(" forced bit positions:", stats["forced_bits"])

```

genesis

```

F_t matches h_t + W_t: True
forced bits from carry trace alone: 15
ambiguous bits remaining: 17
forced bit positions: [6, 8, 13, 15, 17, 18, 19, 20, 21, 23, 24, 25, 26, 27, 31]

```

block_328734

```

F_t matches h_t + W_t: True
forced bits from carry trace alone: 15
ambiguous bits remaining: 17
forced bit positions: [2, 4, 5, 6, 8, 10, 14, 18, 20, 22, 23, 26, 28, 29, 30]

```

What this notebook currently establishes

- SHA-256 round geometry is sparse and largely reversible locally.
- The remaining local ambiguity collapses into a fused wall.
- The final-add carry restoration identity works exactly.
- The Sziklai differential invariant verifies over random tests.
- Admissible side geometry produces a nontrivial residual field.
- Real Bitcoin headers can be tracked backward over bounded depths with best-first residual navigation.
- The fused wall already has a bitwise decoder structure.

What remains open

- globally injective bundle design,
- a proven admissible future-cost lower bound,
- full fused-wall split closure under side data alone,
- a tail-to-vestibule bridge,
- blind full-depth deterministic recovery.